Project 1

Verilog Implementation of Spartan 6 - DSP48A1

Digital Design & Verification Diploma

Delivered on August 4th, 2025

Student Name: Mahmoud Magdy [LinkedIn]

Electronics Engineering Student, Beni Suef University

Ex-Business Developer at P-Vita & SIGMA Elevator

Ex-Visiting Student at Southern Illinois University Edwardsville, USA [Read my story]

Table of Content:

1. PROJECT DESCRIPTION:

The Spartan-6 family offers a high ratio of DSP48A1 slices to logic, making it ideal for math intensive applications. Design DSP48A1 slice of the spartan6 FPGAs. The testbench for this design can be challenging so use directed test patterns whenever needed to verify the design and either have expected value to compare with in the testbench or check the result from the waveforms. Use do file to run the Questasim flow. Use Vivado to go through the design flow running elaboration, synthesis, implementation making sure that there are no design check errors during the design flow.

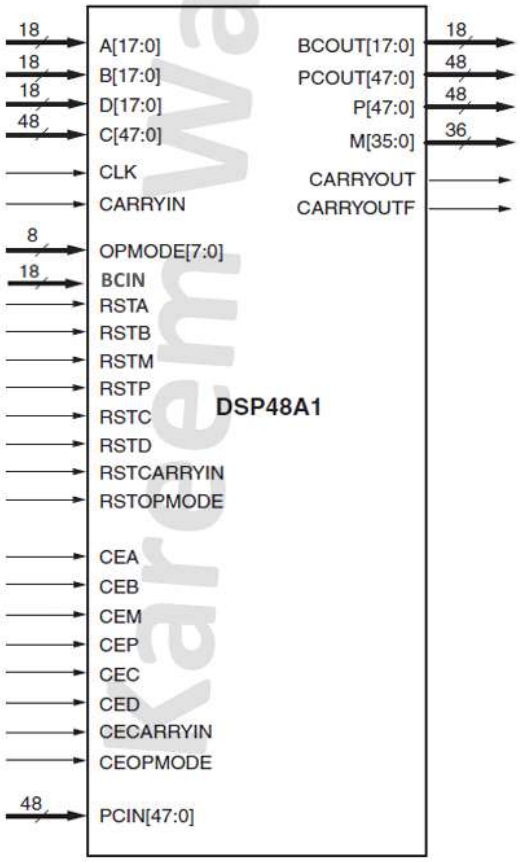

Figure 1 shows the I/O ports of the DSP48A1 to be implemented

From the Inside Schematic of the DSP48A1, shown in Fig. 2, The design can be simplified by implementing a “ff_mux” Module highlighted in Yellow, and a “mux4_1” Module as they are repeated multiple times in the Design.

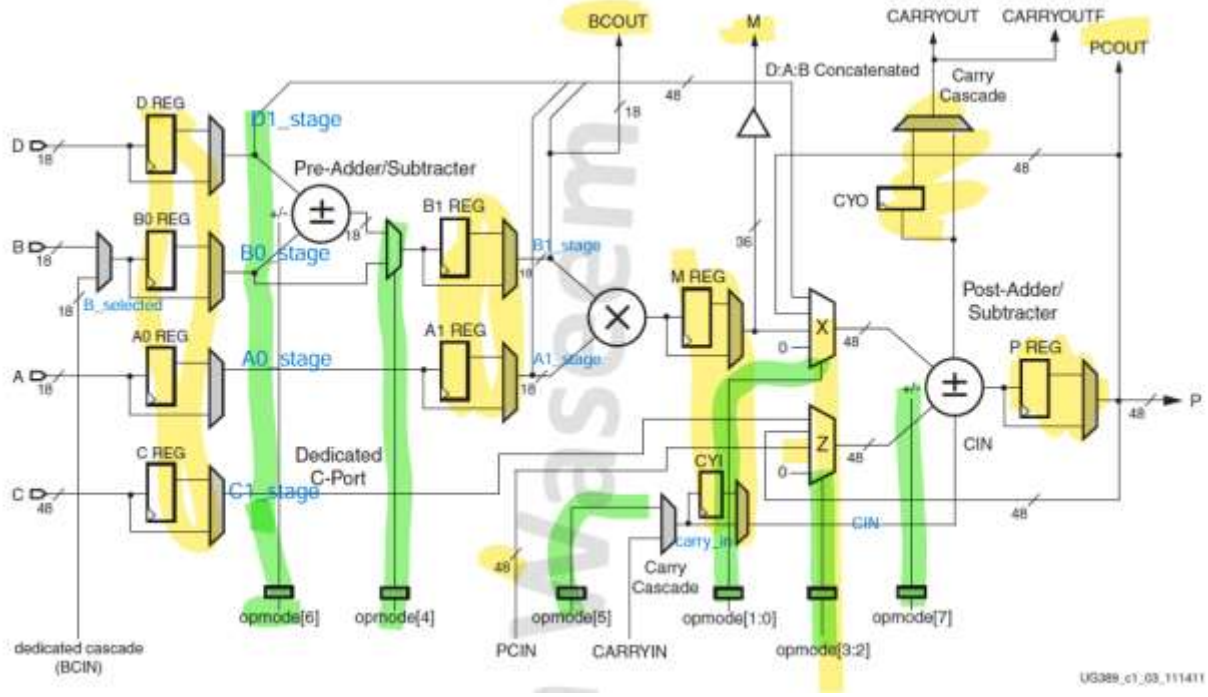

Figure 2 shows the inside schematic of the DSP48A1

The Internal Signals, written in the Blue font as shown in Fig.2, have been implemented to facilitate the flow of signals from Inputs to Outputs through the Datapath correctly.

2. RTL CODE

a. ff_mux.v

```
module ff_mux #(
    parameter reset_type = "SYNC", // takes "SYNC" or "ASYNC"
    parameter WIDTH = 18
) (
    input [WIDTH-1:0] in,
    output reg [WIDTH-1:0] out,
    input sel,
    input CLK,
    input Enable,
    input RST
);
    // Synchronous reset handling
    always @(posedge CLK) begin
        if (reset_type == "SYNC") begin
            if (RST) begin
                out = 0; // Reset the register
            end else if (Enable) begin
                if (sel) begin
                    out <= in;
                end else begin
                    out = in;
                end
            end else begin
                out = out; // Hold previous value if not enabled
            end
        end
    end
    // Asynchronous reset handling
    always @(posedge CLK or posedge RST) begin
        if (reset_type == "ASYNC") begin
            if (RST) begin
                out <= 0; // Reset output
            end else if (Enable) begin
                if (sel) begin
                    out <= in; // Update output with the registered value
                end else begin
                    out = in; // Directly assign input to output if not
selected
                end
            end else begin
                out = out; // Hold previous value if not enabled
            end
        end
    end
endmodule
```

b. mux4_1.v

```
module mux4_1 (
    input [47:0] in1,
    input [47:0] in2,
    input [47:0] in3,
    output reg [47:0] out,
    input [1:0] sel
);
    always @(*) begin
        case (sel)
            2'b00: out = 0;
            2'b01: out = in1;
            2'b10: out = in2;
            2'b11: out = in3;
        endcase
    end
endmodule
```

c. dsp.v TOP_MODULE

```
module dsp #(
    parameter A0REG = 0,
    parameter A1REG = 1,
    parameter B0REG = 0,
    parameter B1REG = 1,
    parameter CREG = 1,
    parameter DREG = 1,
    parameter MREG = 1,
    parameter PREG = 1,
    parameter CARRYINREG = 1,
    parameter CARRYOUTREG = 1,
    parameter OPMODEREG = 1,
    parameter CARRYINSEL = "OPMODE5", // takes "OPMODE5" or "CARRYIN"
    parameter B_INPUT = "DIRECT", // takes "DIRECT" or "CASCADE"
    parameter RSTTYPE = "SYNC" // takes "SYNC" or "ASYNC"
) (
    // Data Ports
    input [17:0] A,
    input [17:0] B,
    input [47:0] C,
    input [17:0] D,
    input CARRYIN,
    output [35:0] M,
    output [47:0] P,
    output CARRYOUT,
    output CARRYOUTF,

    // Control Input Ports
    input CLK,
    input [0:7] OPMODE,

    // Clock Enable Input Ports
    input CEA,
    input CEB,
    input CEC,
    input CECARRYIN,
    input CED,
    input CEM,
    input CEOPMODE,
    input CEP,

    // Reset Input Ports
    input RSTA,
    input RSTB,
    input RSTC,
    input RSTCARRYIN,
    input RSTD,
    input RSTM,
    input RSTOPMODE,
    input RSTP,

    // Cascade Ports
```

```
    input [17:0] BCIN,
    output [17:0] BCOUT,
    input [47:0] PCIN,
    output [47:0] PCOUT
);
    // Internal signals
    wire [7:0] OPMODE_reg;
    wire [17:0] A0_stage;
    wire [17:0] B0_stage;
    wire [47:0] C1_stage;
    wire [17:0] D1_stage;
    wire [17:0] B_selected;
    reg [17:0] pre_adder;
    wire [17:0] pre_adder_mux;
    wire [17:0] A1_stage;
    wire [17:0] B1_stage;
    wire [35:0] multiplier_out;
    wire carry_in;
    wire CIN;
    wire [47:0] M_extended;
    wire [47:0] concatenated;
    wire [47:0] z_out;
    wire [47:0] x_out;
    reg [47:0] post_adder;
    reg post_adder_co;

    // B input selection
    assign B_selected = (B_INPUT == "DIRECT") ? B : (B_INPUT == "CASCADE")
? BCIN : 0;

    // First Stage in Pipeline
    ff_mux #(.WIDTH(8)) OPMODE_stage (.in(OPMODE), .out(OPMODE_reg),
.sel(OPMODEREG),
    .CLK(CLK), .Enable(CEOPMODE), .RST(RSTOPMODE));
    ff_mux D_stage (.in(D), .out(D1_stage), .sel(DREG), .CLK(CLK),
.Enable(CED), .RST(RSTD));
    ff_mux A_stage (.in(A), .out(A0_stage), .sel(A0REG), .CLK(CLK),
.Enable(CEA), .RST(RSTA));
    ff_mux B_stage (.in(B_selected), .out(B0_stage), .sel(B0REG),
.CLK(CLK), .Enable(CEB), .RST(RSTB));
    ff_mux #(.WIDTH(48)) C_stage (.in(C), .out(C1_stage), .sel(CREG),
.CLK(CLK), .Enable(CEC), .RST(RSTC));

    // Pre-Adder/Subtraction
    always @(posedge CLK) begin
      case (OPMODE_reg[6])
        1'b0: pre_adder = D1_stage + B0_stage;
        1'b1: pre_adder = D1_stage - B0_stage;
      endcase
    end

    // MUX for Pre-Adder Output
    assign pre_adder_mux = (OPMODE_reg[4]) ? pre_adder : B0_stage;
```

```
    // Second Stage in Pipeline
    ff_mux A1_stage_ff (.in(A0_stage), .out(A1_stage), .sel(A1REG),
.CLK(CLK), .Enable(CEA), .RST(RSTA));
    ff_mux B1_stage_ff (.in(pre_adder_mux), .out(B1_stage), .sel(B1REG),
.CLK(CLK), .Enable(CEB), .RST(RSTB));

    // Cascade output for Port B
    assign BCOUT = B1_stage;

    // Multiplier Out
    assign multiplier_out = A1_stage * B1_stage;
    // Carry IN Cascade
    assign carry_in = (CARRYINSEL == "OPMODE5") ? OPMODE_reg[5] :
(CARRYINSEL == "CARRYIN") ? CARRYIN : 0;

    // Third Stage in Pipeline
    ff_mux #(.WIDTH(36)) M_stage (.in(multiplier_out), .out(M),
.sel(MREG), .CLK(CLK), .Enable(CEM), .RST(RSTM));
    ff_mux #(.WIDTH(1)) CYI (.in(carry_in), .out(CIN), .sel(CARRYINREG),
.CLK(CLK), .Enable(CECARRYIN), .RST(RSTCARRYIN));
    assign M_extended = {12'b0, M};

    // Z MUX out
    mux4_1 z_mux (.in3(C1_stage), .in2(PCOUT), .in1(PCIN),
.sel(OPMODE_reg[3:2]), .out(z_out));
    // X MUX out
    assign concatenated = {D1_stage[11:0], A1_stage[17:0],
B1_stage[17:0]};
    mux4_1 x_mux (.in3(concatenated), .in2(PCOUT), .in1(M_extended),
.sel(OPMODE_reg[1:0]), .out(x_out));

    // Post-Adder Subtractor
    always @(posedge CLK) begin
      case (OPMODE_reg[7])
      1'b0: {post_adder_co, post_adder} = x_out + z_out + CIN;
      1'b1: {post_adder_co, post_adder} = z_out - (x_out + CIN);
      endcase
    end

    // CASCADE CARRY OUT
    ff_mux #(.WIDTH(1)) CYO (.in(post_adder_co), .out(CARRYOUT),
.sel(CARRYOUTREG),
    .CLK(CLK), .Enable(CECARRYIN), .RST(RSTCARRYIN));
    assign CARRYOUTF = CARRYOUT;

    // Output P
    ff_mux #(.WIDTH(48)) P_out (.in(post_adder), .out(P), .sel(PREG),
.CLK(CLK), .Enable(CEP), .RST(RSTP));
    assign PCOUT = P;

endmodule
```

3. TESTBENCH CODE (RESULTS IN AFTER NEXT SECTION)

dsp_tb.v

```
module dsp_tb();
    parameter A0REG = 0;
    parameter A1REG = 1;
    parameter B0REG = 0;
    parameter B1REG = 1;
    parameter CREG = 1;
    parameter DREG = 1;
    parameter MREG = 1;
    parameter PREG = 1;
    parameter CARRYINREG = 1;
    parameter CARRYOUTREG = 1;
    parameter OPMODEREG = 1;
    parameter CARRYINSEL = "OPMODE5"; // takes "OPMODE5" or "CARRYIN"
    parameter B_INPUT = "DIRECT"; // takes "DIRECT" or "CASCADE"
    parameter RSTTYPE = "SYNC"; // takes "SYNC" or "ASYNC"

    reg [17:0] A;
    reg [17:0] B;
    reg [17:0] D;
    reg [47:0] C;
    reg CARRYIN;
    wire [35:0] M;
    wire [47:0] P;
    wire CARRYOUT;
    wire CARRYOUTF;

    reg CLK;
    reg [0:7] OPMODE;

    reg CEA;
    reg CEB;
    reg CEC;
    reg CECARRYIN;
    reg CED;
    reg CEM;
    reg CEOPMODE;
    reg CEP;

    reg RSTA;
    reg RSTB;
    reg RSTC;
    reg RSTCARRYIN;
    reg RSTD;
    reg RSTM;
    reg RSTOPMODE;
    reg RSTP;

    reg [17:0] BCIN;
    wire [17:0] BCOUT;
```

```
reg [47:0] PCIN;
wire [47:0] PCOUT;

// instatiate the DSP module
dsp #(
    .A0REG(A0REG),
    .A1REG(A1REG),
    .B0REG(B0REG),
    .B1REG(B1REG),
    .CREG(CREG),
    .DREG(DREG),
    .MREG(MREG),
    .PREG(PREG),
    .CARRYINREG(CARRYINREG),
    .CARRYOUTREG(CARRYOUTREG),
    .OPMODEREG(OPMODEREG),
    .CARRYINSEL(CARRYINSEL),
    .B_INPUT(B_INPUT),
    .RSTTYPE(RSTTYPE)
) uut (
    .A(A),
    .B(B),
    .C(C),
    .D(D),
    .CARRYIN(CARRYIN),
    .M(M),
    .P(P),
    .CARRYOUT(CARRYOUT),
    .CARRYOUTF(CARRYOUTF),

    // Control Input Ports
    .CLK(CLK),
    .OPMODE(OPMODE),

    // Clock Enable Input Ports
    .CEA(CEA),
    .CEB(CEB),
    .CEC(CEC),
    .CECARRYIN(CECARRYIN),
    .CED(CED),
    .CEM(CEM),
    .CEOPMODE(CEOPMODE),
    .CEP(CEP),

    // Reset Input Ports
    .RSTA(RSTA),
    .RSTB(RSTB),
    .RSTC(RSTC),
    .RSTCARRYIN(RSTCARRYIN),
    .RSTD(RSTD),
    .RSTM(RSTM),
    .RSTOPMODE(RSTOPMODE),
    .RSTP(RSTP),
```

```
    // Cascade Ports
    .BCIN(BCIN),
    .BCOUT(BCOUT),
    .PCIN(PCIN),
    .PCOUT(PCOUT)
);

// Clock generation
initial begin
    CLK = 0;
    forever #5 CLK = ~CLK; // 100 MHz clock
end

// Testbench stimulus
initial begin
    // Assert all active-high reset signals by setting them to 1
    RSTA = 1;
    RSTB = 1;
    RSTC = 1;
    RSTCARRYIN = 1;
    RSTD = 1;
    RSTM = 1;
    RSTOPMODE = 1;
    RSTP = 1;
    A = $random;
    B = $random;
    C = $random;
    D = $random;
    CARRYIN = $random;
    OPMODE = $random;
    CEA = $random;
    CEB = $random;
    CEC = $random;
    CECARRYIN = $random;
    CED = $random;
    CEM = $random;
    CEOPMODE = $random;
    CEP = $random;
    BCIN = $random;
    PCIN = $random;
    @ (negedge CLK); // Wait for a clock edge
    // Self-Checking to verify all outputs are zero
    if (P !== 0 || M !== 0 || CARRYOUT !== 0 || CARRYOUTF !== 0) begin
        $display("Test failed: Outputs are not zero after reset.");
    end else begin
        $display("Test passed: Outputs are zero after reset.");
    end
    // Deassert reset signals
    RSTA = 0;
    RSTB = 0;
    RSTC = 0;
    RSTCARRYIN = 0;
    RSTD = 0;
    RSTM = 0;
```

```
        RSTOPMODE = 0;
        RSTP = 0;
        // assert all clock enable signals to validate the functionality
of the subsequent DSP paths
        CEA = 1;
        CEB = 1;
        CEC = 1;
        CECARRYIN = 1;
        CED = 1;
        CEM = 1;
        CEOPMODE = 1;
        CEP = 1;
        @ (negedge CLK);

        // Verify DSP Path 1
        A = 20;
        B = 10;
        C = 350;
        D = 25;
        OPMODE = 8'b11011101;
        BCIN = $random;
        PCIN = $random;
        CARRYIN = $random;
        repeat (6) @(negedge CLK);
        // Self-Checking outputs
        if (P !== 'h32 || PCOUT !== 'h32 || M !== 'h12c || BCOUT !== 'hf||
CARRYOUT !== 0 || CARRYOUTF !== 0) begin
            $display("Test failed: DSP Path 1 outputs are incorrect.");
        end else begin
            $display("Test passed: DSP Path 1 outputs are correct.");
        end

        // Verify DSP Path 2
        A = 20;
        B = 10;
        C = 350;
        D = 25;
        OPMODE = 8'b00010000;
        BCIN = $random;
        PCIN = $random;
        CARRYIN = $random;
        repeat (5) @(posedge CLK);
        // Self-Checking outputs
        if (P !== 'h0 || PCOUT !== 'h0 || M !== 'h2bc || BCOUT !== 'h23 ||
CARRYOUT !== 0 || CARRYOUTF !== 0) begin
            $display("Test failed: DSP Path 2 outputs are incorrect.");
        end else begin
            $display("Test passed: DSP Path 2 outputs are correct.");
        end

        // Verify DSP Path 3
        OPMODE = 8'b00001010;
        A = 20;
        B = 10;
```

```
        C = 350;
        D = 25;
        BCIN = $random;
        PCIN = $random;
        CARRYIN = $random;
        repeat (5) @(posedge CLK);
        // Self-Checking outputs
        if (P !== 'h0 || PCOUT !== 'h0 || M !== 'hc8 || BCOUT !== 'ha ||
CARRYOUT !== 0 || CARRYOUTF !== 0) begin
            $display("Test failed: DSP Path 3 outputs are incorrect.");
        end else begin
            $display("Test passed: DSP Path 3 outputs are correct.");
        end

        // Verify DSP Path 4
        OPMODE = 8'b10100111;
        A = 5;
        B = 6;
        C = 350;
        D = 25;
        PCIN = 3000;
        BCIN = $random;
        CARRYIN = $random;
        repeat (5) @(posedge CLK);
        // Self-Checking outputs
        if (P !== 'hfe6fffec0bb1 || PCOUT !== 'hfe6fffec0bb1 || M !== 'h1e
|| BCOUT !== 'h6 || CARRYOUT !== 1 || CARRYOUTF !== 1) begin
            $display("Test failed: DSP Path 4 outputs are incorrect.");
        end else begin
            $display("Test passed: DSP Path 4 outputs are correct.");
        end

        $finish; // End simulation
    end

endmodule
```

4. DO FILE

run.do

```
vlib work
vlog dsp.v
vlog dsp_tb.v
vlog ff_mux.v
vlog mux4_1.v

vsim -voptargs=+acc work.dsp_tb
add wave *
run -all
#quit -sim
```

5. QUESTASIM SNIPPETS

a. Verifying Reset Operation SUCCESSFUL

Assert all active-high reset signals by setting them to 1.

Drive remaining inputs with arbitrary (random) values.

Wait for the negative edge of the clock.

Add a condition for self-checking to verify that all outputs are zero.

Deassert all reset signals and assert all clock enable signals to validate the functionality of the subsequent DSP paths.

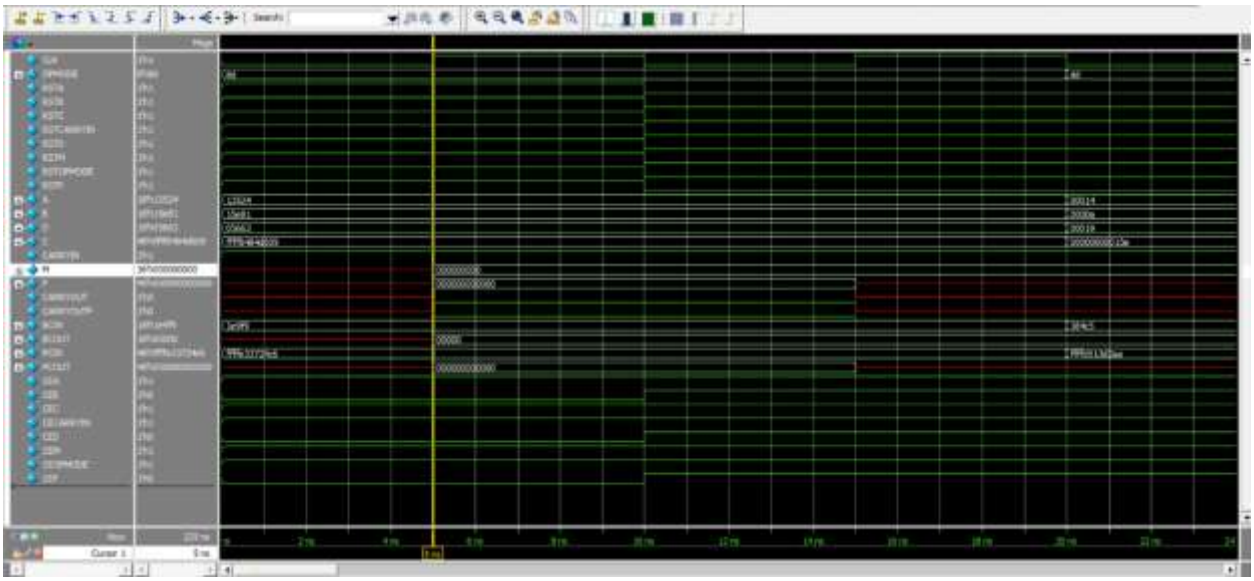

Figure 3 shows correct Functioning of reset operation

b. Verify DSP Path 1 SUCCESSFUL

- This test path evaluates the pre-subtractor, allowing it to propagate and post-subtractor stages by enabling the multiplier output to route through Mux X and the C-port through Mux Z, corresponding to OPMODE = 8'b11011101.
- Apply the following input values: A = 20, B = 10, C = 350, and D = 25.
- Drive BCIN, PCIN, and CARRYIN with arbitrary (random) values.
- Wait for four negative clock edges, as the data propagates through four flip-flops (DREG, B1REG, MREG, and PREG), as illustrated in Figure (1).
- The expected outputs are: BCOUT = 'hf, M = 'h12c, P = PCOUT = 'h32, and CARRYOUT = CARRYOUTF = 0. • Add a condition for self-checking to verify that the design outputs with the expected outputs.

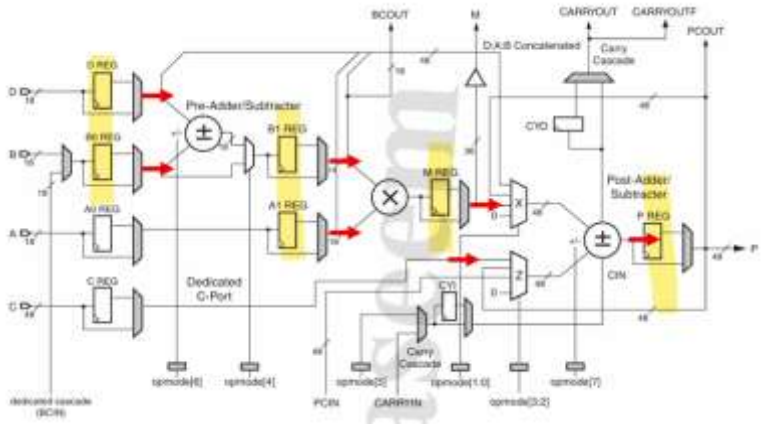

Figure 4 shows Path 1 Data Flow

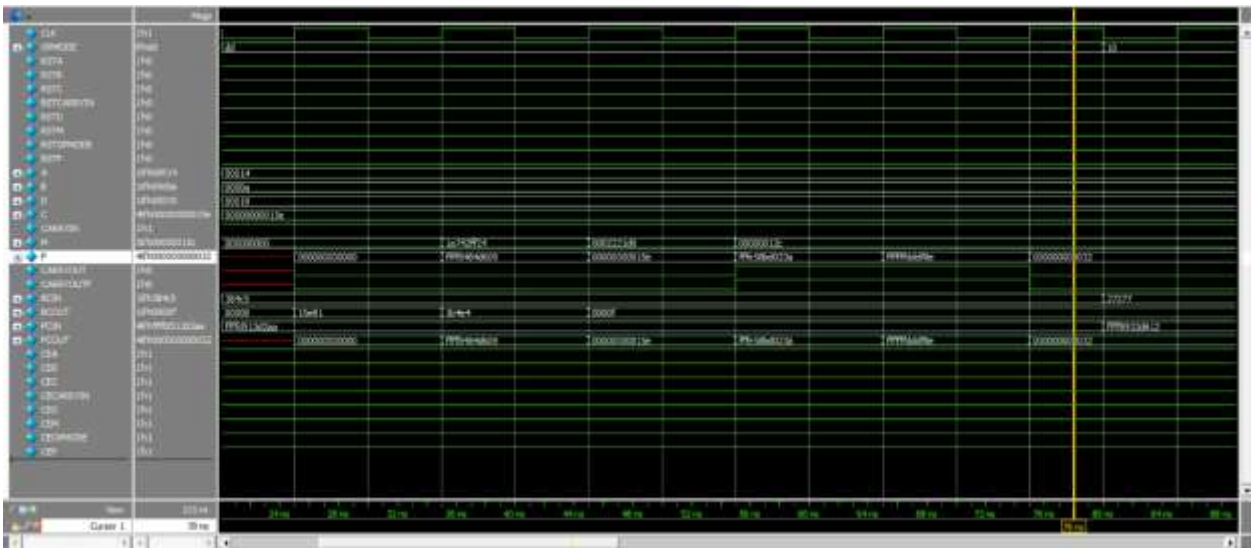

Figure 5 shows correct Data Flow in Path 1

c. Verifying DSP Path 2 SUCCESSFUL

- This test path evaluates the pre-addition allowing it to propagate and post-addition stages by enabling the zeros to route through Mux X and Mux Z, corresponding to OPMODE = 8'b00010000.
- Apply the following input values: A = 20, B = 10, C = 350, and D = 25.
- Drive BCIN, PCIN, and CARRYIN with arbitrary (random) values.
- Wait for three negative edges, as the data propagates through three flip-flops (DREG, B1REG, MREG) as longest path and also pass to PREG in parallel.
- The expected outputs are: BCOUT = 'h23, M = 'h2bc, P = PCOUT = 0, and CARRYOUT = CARRYOUTF = 0.
- Add a condition for self-checking to verify that the design outputs with the expected outputs.

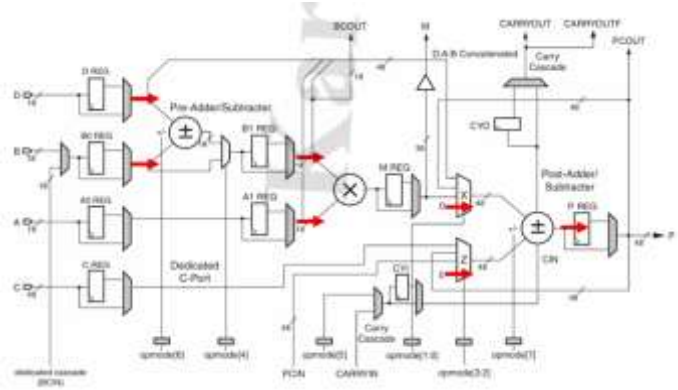

Figure 6 shows Data Flow in Path 2

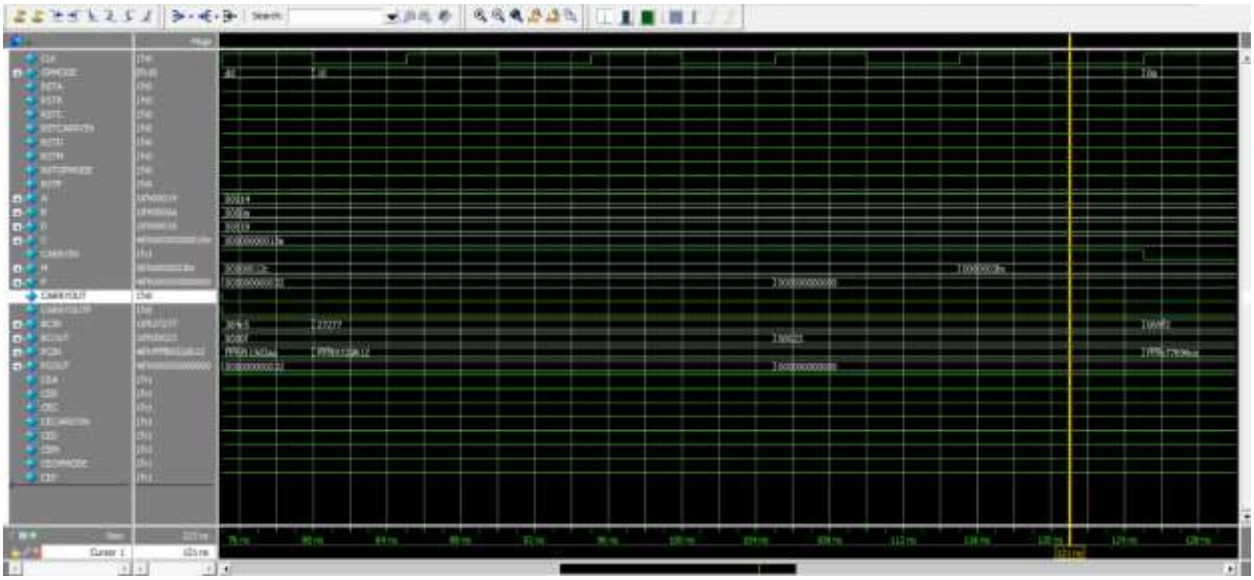

Figure 7 shows correct output from Path 2

d. Verifying DSP Path 3 SUCCESSFUL

- This test path has no pre-addition/subtractor (doesn't allow it to propagate) and allows post-addition stages by enabling the P feedback to route through Mux X and Mux Z, corresponding to OPMODE = 8'b00001010.
- Apply the following input values: A = 20, B = 10, C = 350, and D = 25.
- Drive BCIN, PCIN, and CARRYIN with arbitrary (random) values.
- Wait for three negative edges, as the data propagates through three flip-flops (B1REG, MREG, PREG) as longest path and also pass to PREG in parallel, as illustrated in Figure (3).
- The expected outputs are: BCOUT = 'ha, M = 'hc8, P = PCOUT which is the past Value of P, and CARRYOUT = CARRYOUTF which is the pat value of CARRYOUT.
- Add a condition for self-checking to verify that the design outputs with the expected outputs.

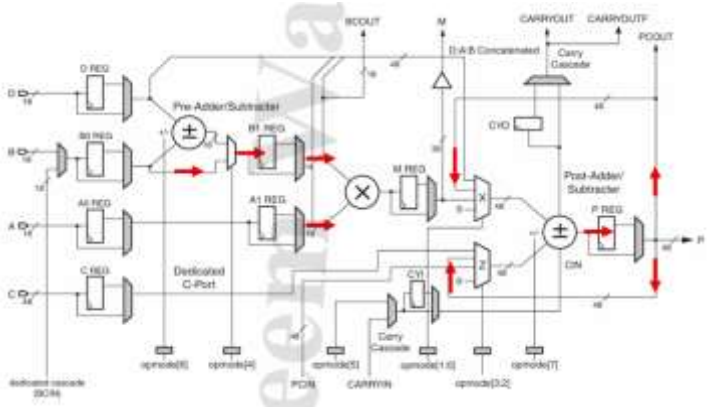

Figure 9 shows Data Flow in Path 3

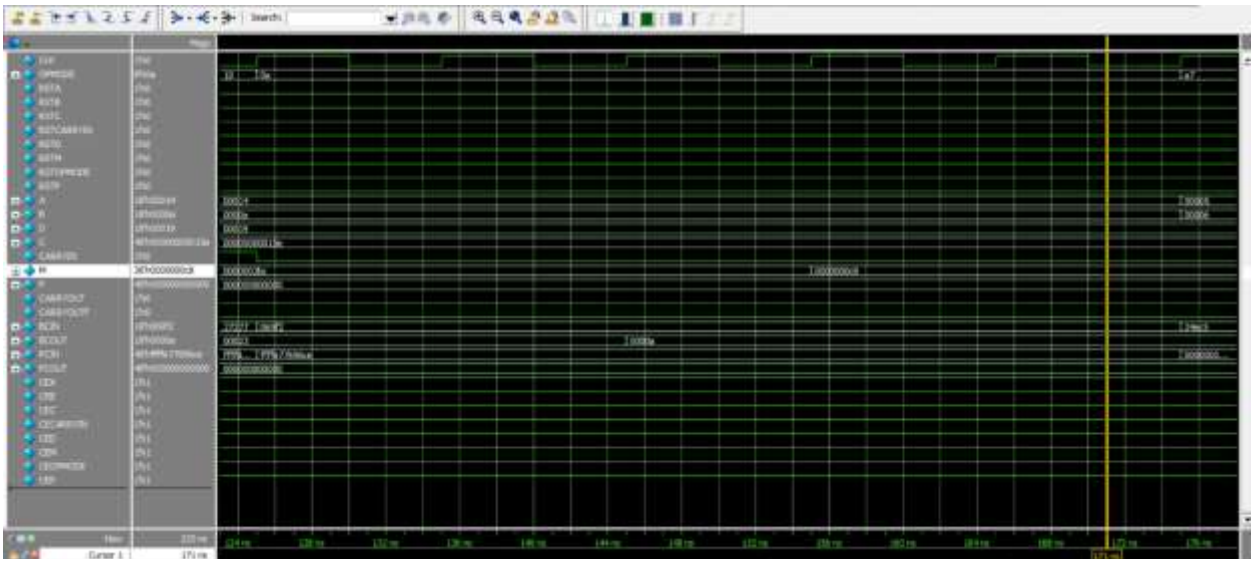

Figure 8 shows correct output through Data Path 3

e. Verifying DSP Path 4 SUCCESSFUL

- This test path has no pre-addition/subtractor (doesn't allow it to propagate) and allows post-subtractor stages by enabling D:A:B Concatenated to route through Mux X and PCIN to Mux Z, corresponding to OPMODE = 8'b10100111.
- Apply the following input values: A = 5, B = 6, C = 350, D = 25 and PCIN = 3000
- Drive BCIN, and CARRYIN with arbitrary (random) values.
- Wait for three negative edges, as the data propagates through three flip-flops (B1REG, MREG, PREG) as longest path as illustrated in Figure (4).
- The expected outputs are: BCOUT = 'h6, M = 'h1e, P = PCOUT = 'hfe6fffec0bb1, and CARRYOUT = CARRYOUTF = 1.

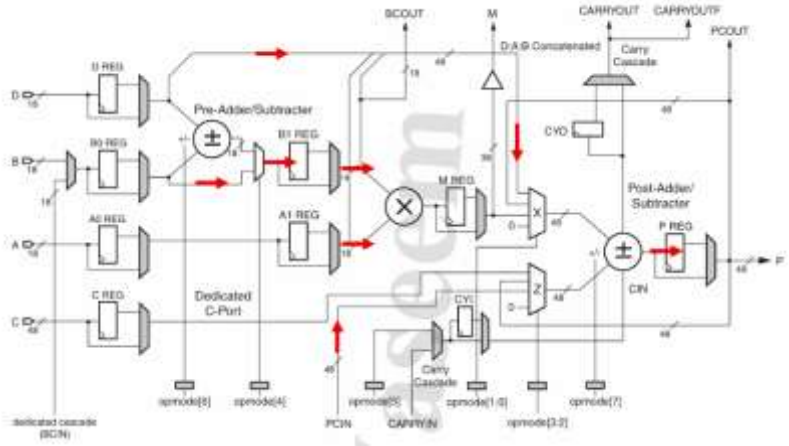

Figure 11 shows Data Flow in Path 4

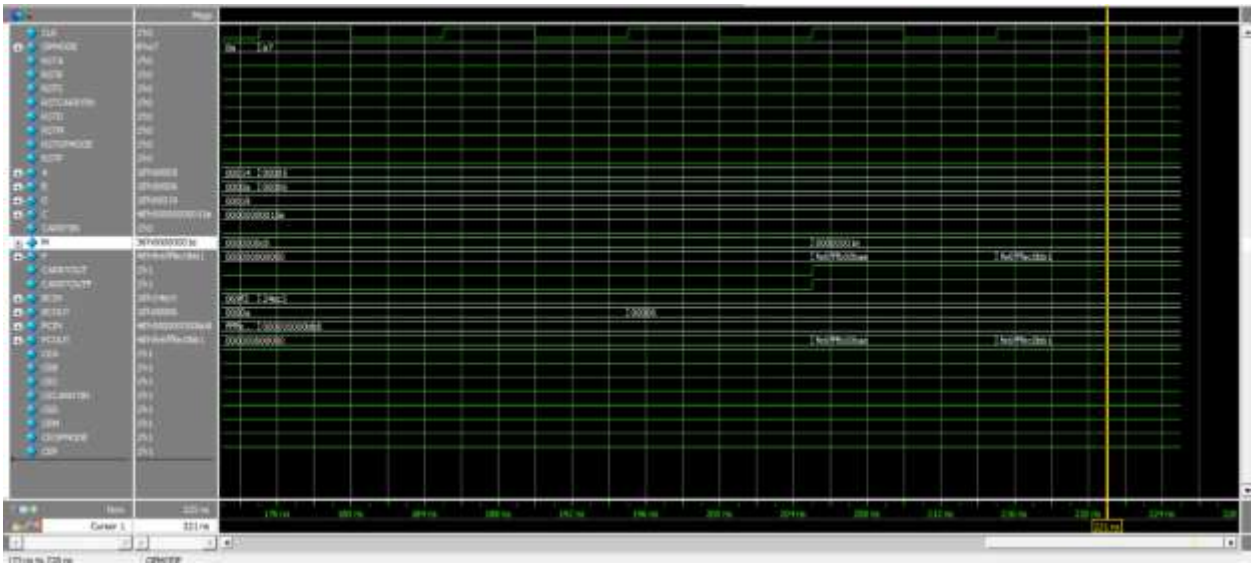

Figure 10 shows correct output from Path 4

f. Output in the Transcript SUCCESSFUL

Successful Self-Checking Operation

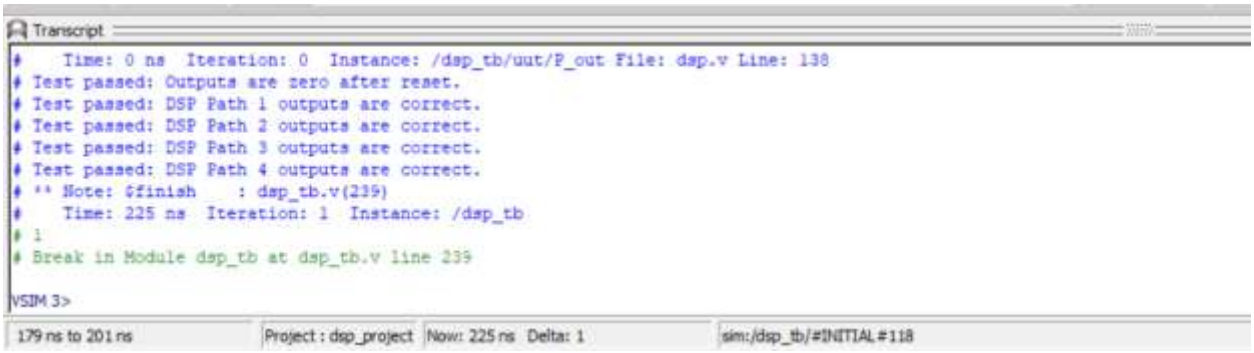

Figure 12 shows the outputs in all cases are the same as expected

6. CONSTRAINT FILE

dsp_constraints.xcd

```
## Clock signal
set_property -dict { PACKAGE_PIN W5   IOSTANDARD LVCMOS33 } [get_ports
CLK]
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5}
[get_ports CLK]


## Configuration options, can be used for all designs
set_property CONFIG_VOLTAGE 3.3 [current_design]
set_property CFGBVS VCCO [current_design]

## SPI configuration mode options for QSPI boot, can be used for all
designs
set_property BITSTREAM.GENERAL.COMPRESS TRUE [current_design]
set_property BITSTREAM.CONFIG.CONFIGRATE 33 [current_design]
set_property CONFIG_MODE SPIx4 [current_design]
```

7. ELABORATION

The generated schematic after Elaboration is shown in Figure 13, and the “Messages” Tab is shown in Figure 14

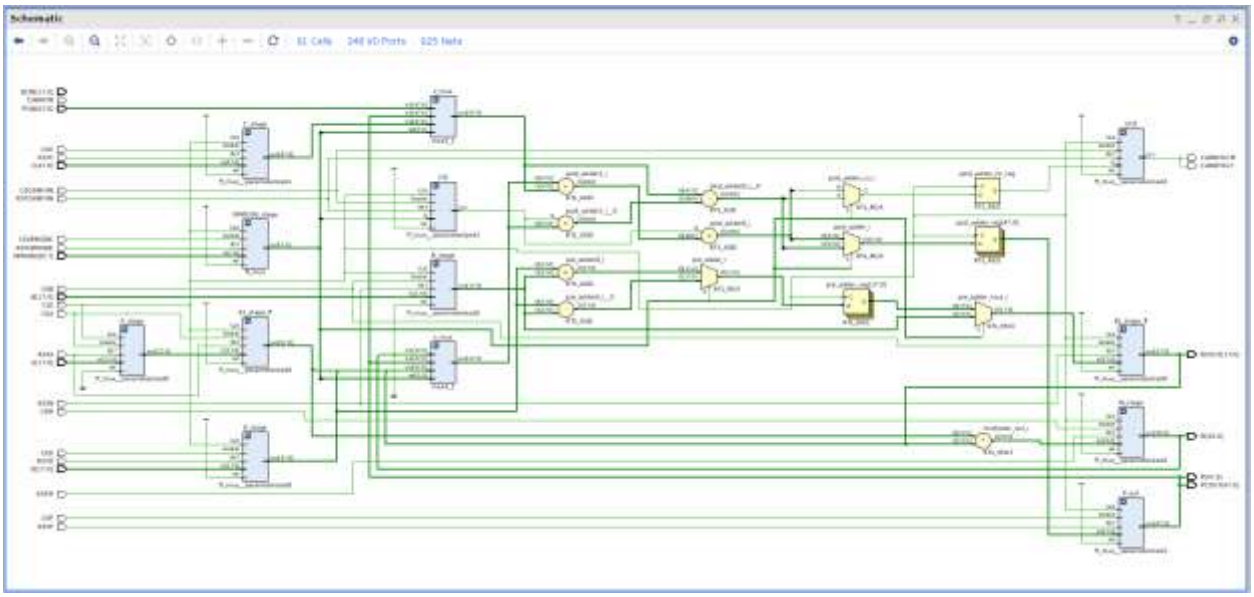

Figure 13 shows the generated DSP48A1 Schematics After Elaboration

No Errors in the Messages Tab

Figure 14 shows the "Messages" Tab after Synthesis with No Errors

8. SYNTHESIS

The generated Schematic, Messages Tab, Utilization Report, and Timing Report are shown in the following figures 15 – 18

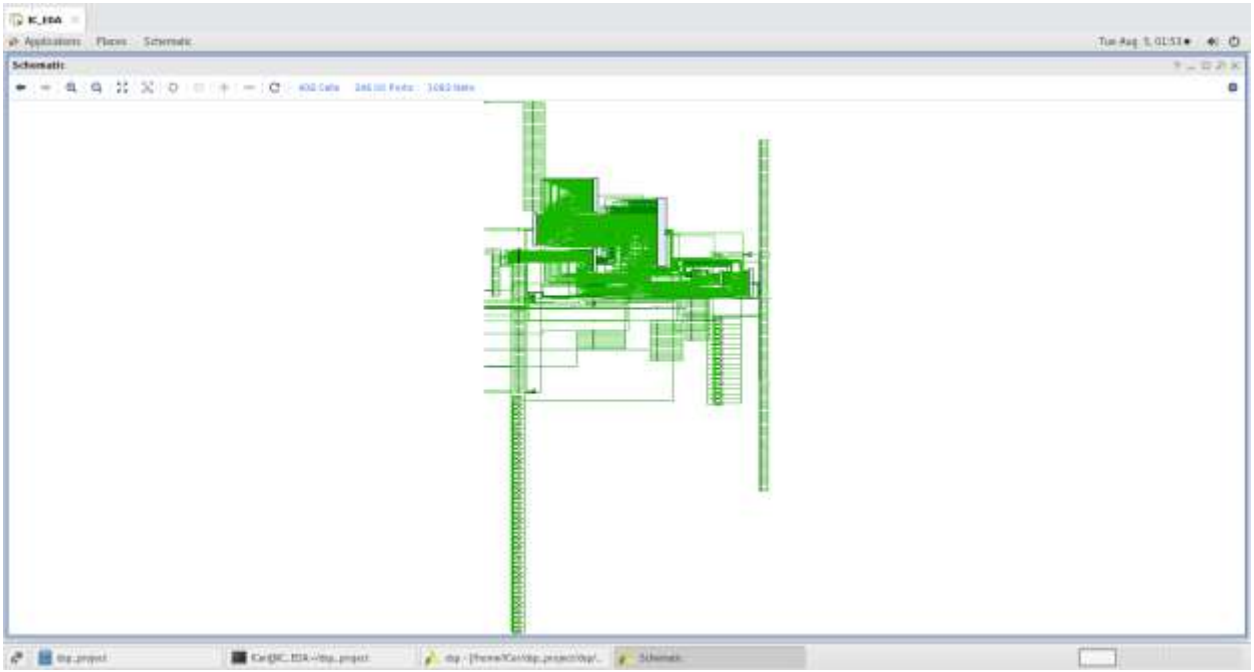

Figure 15 shows the netlist generated after Synthesis

No Errors in the Messages Tab

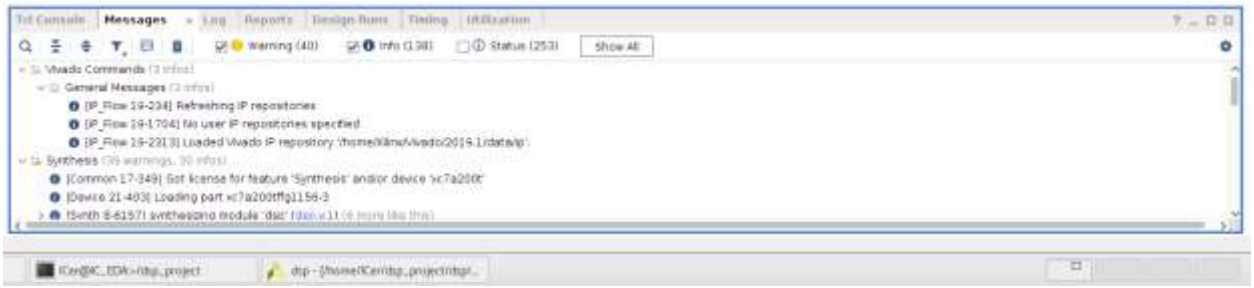

Figure 16 shows the messages tab after synthesis with no Errors

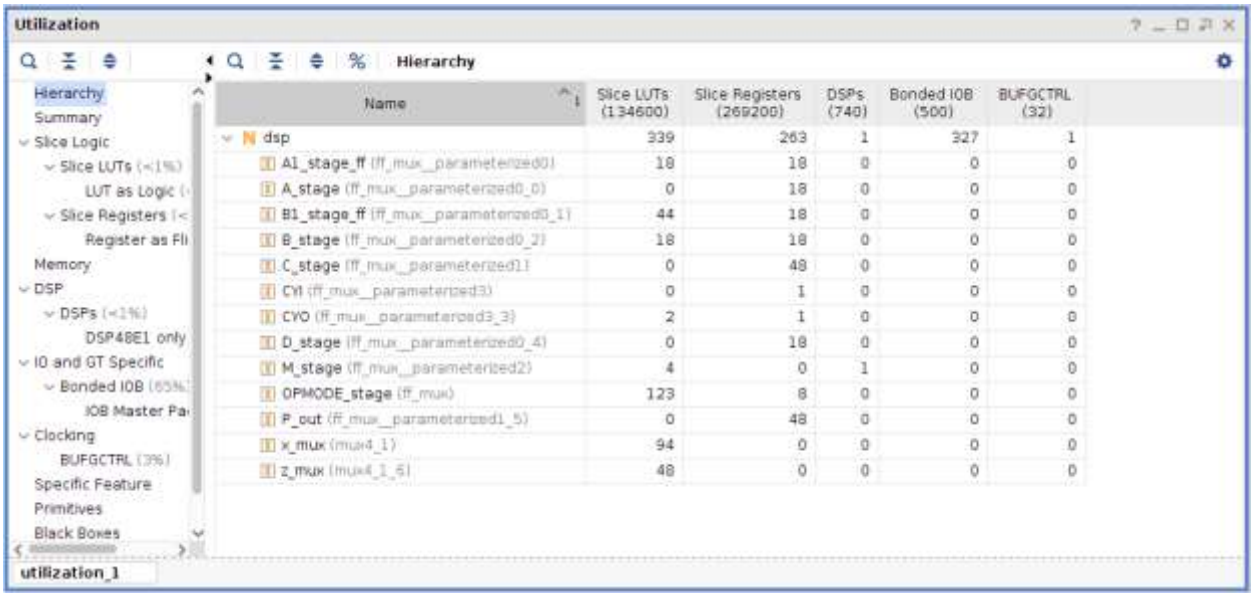

Figure 17 shows the Utilization Report after Synthesis

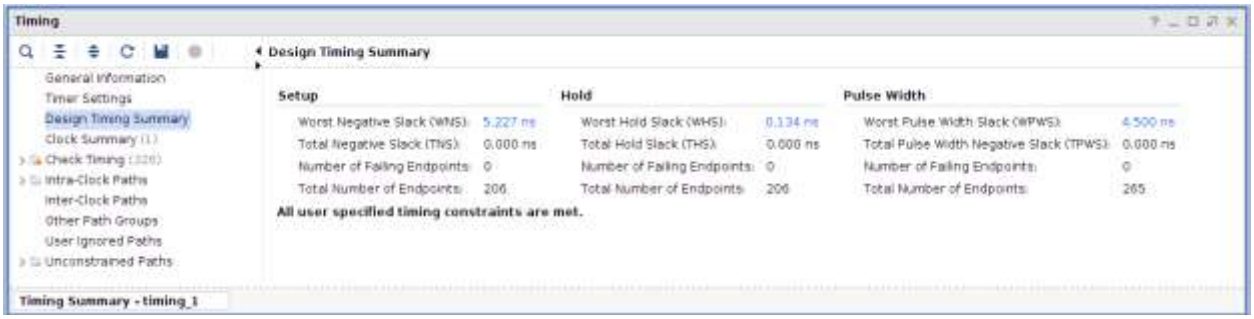

Figure 18 Shows the Timing Report After Synthesis

9. IMPLEMENTATION

The following Figures 19-24 show the schematics generated, Messages Tab, Utilization Report, Timing Report, and Power Report After Implementation of the DSP48A1 Module

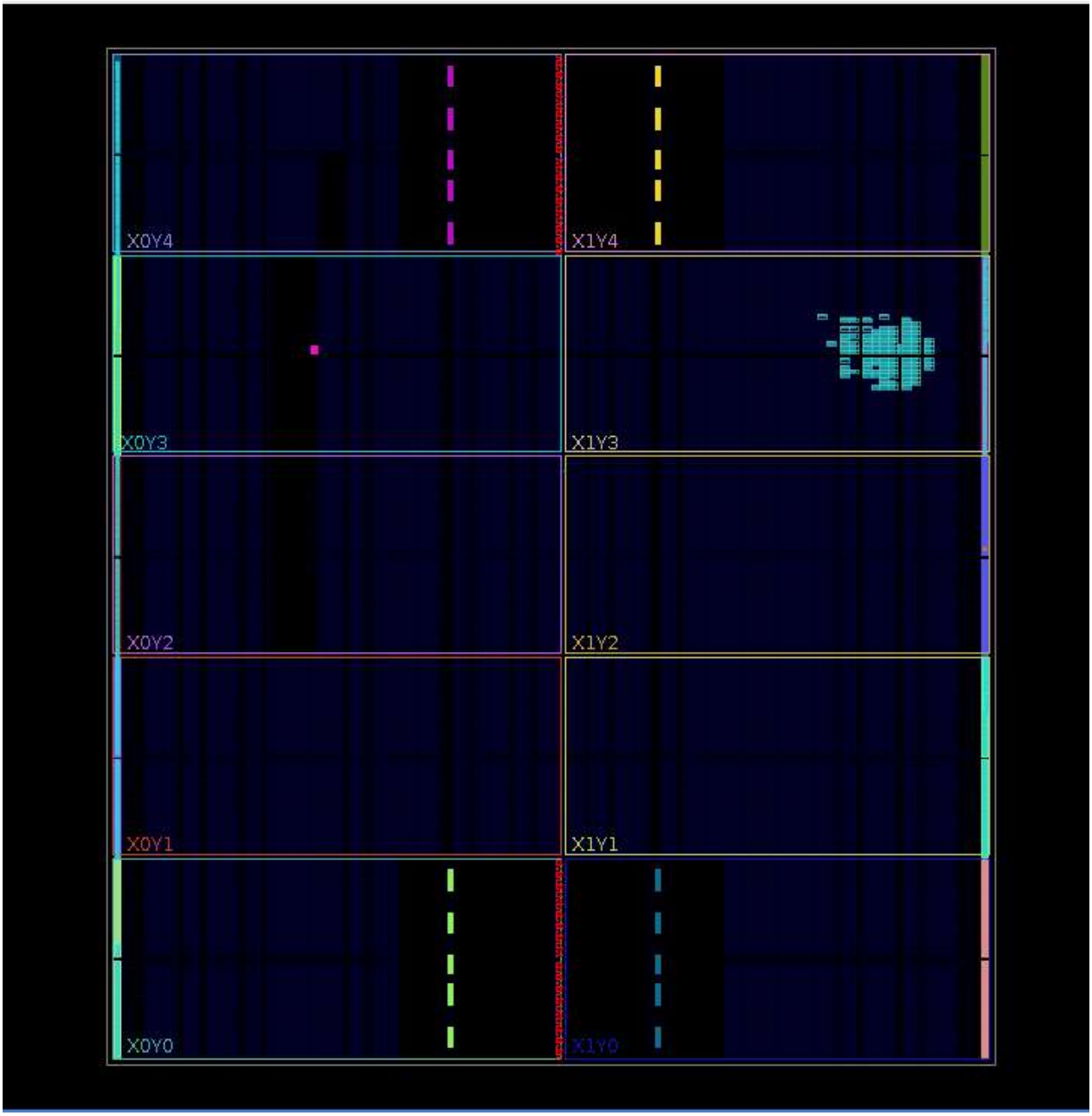

Figure 19 shows the Device Tab after Implementation

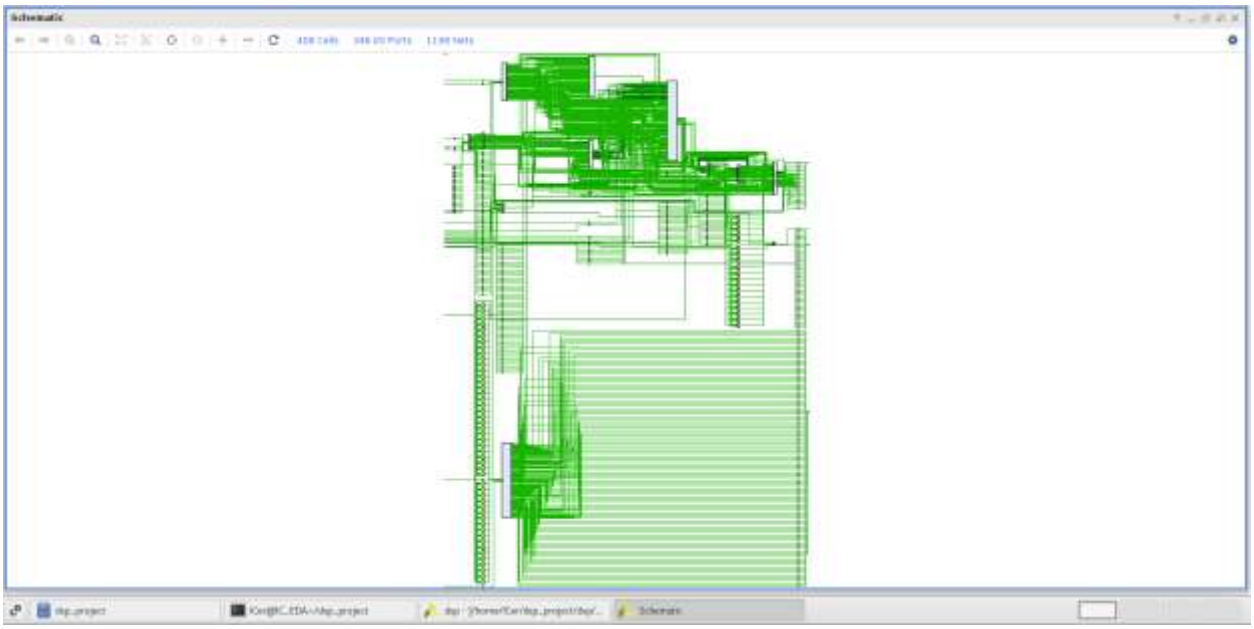

Figure 20 shows the generated schematics after Implementation

No Errors in the Messages Tab

Figure 21 shows the Messages Tab after implementation with No Errors

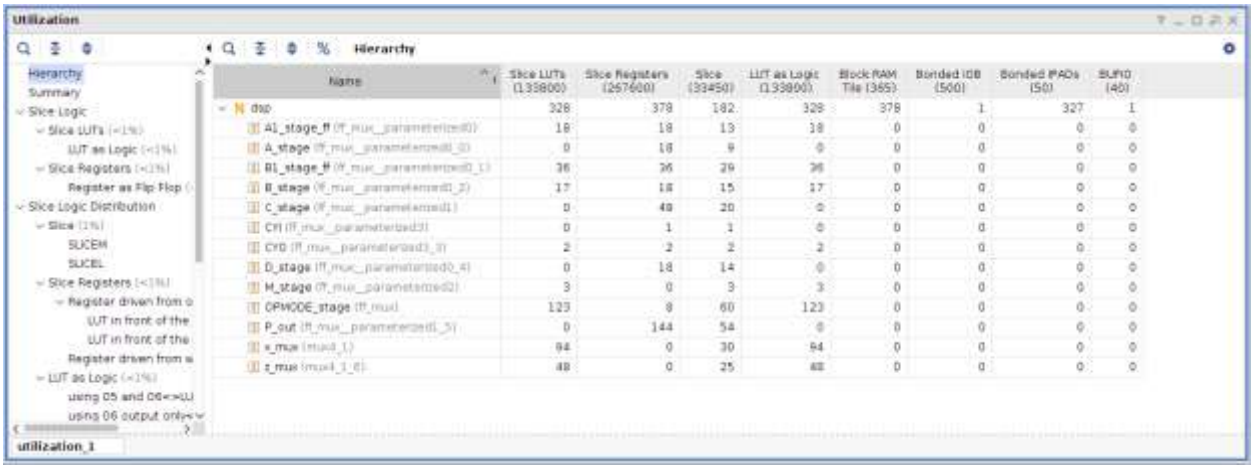

Figure 22 shows the Utilization Report After Implementation

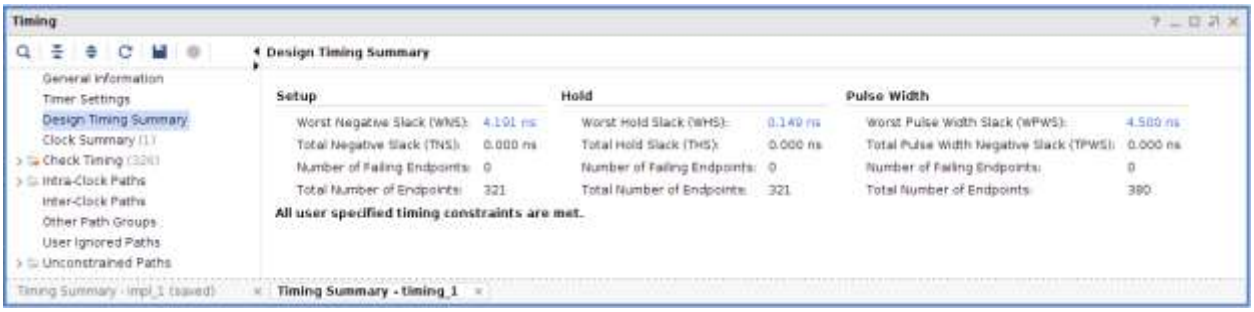

Figure 23 shows the Timing Report After Implementation

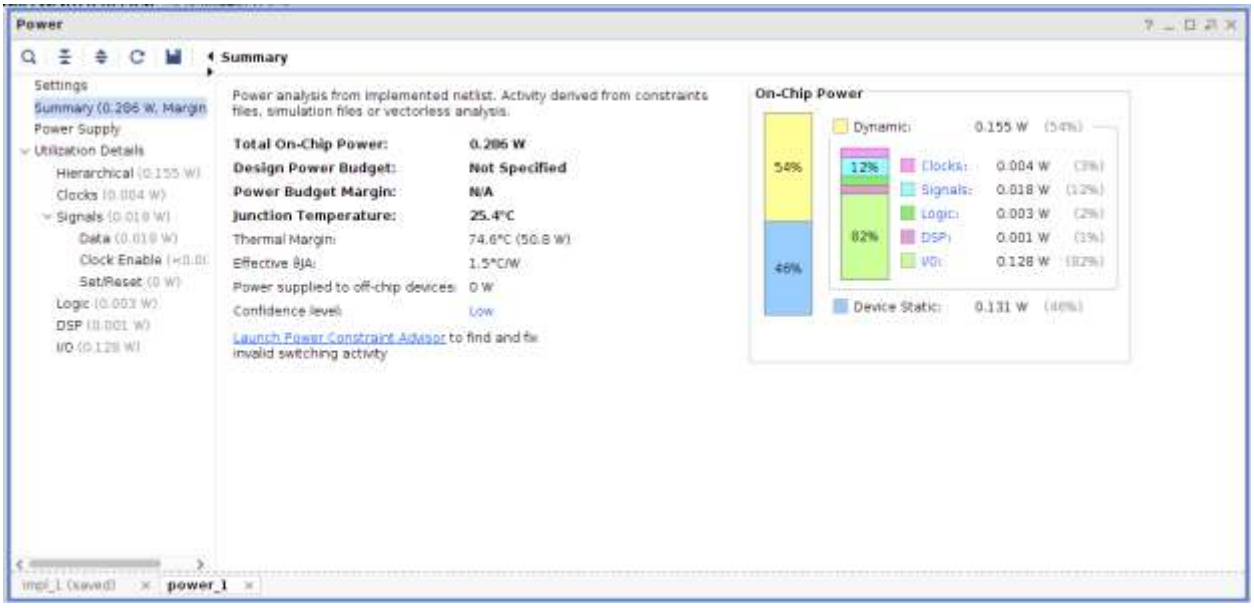

Figure 24 shows the Power Report After Implementation

10.LINTING

The following two figures are the output schematics, and Lint Checks after linting through QuestaLint.

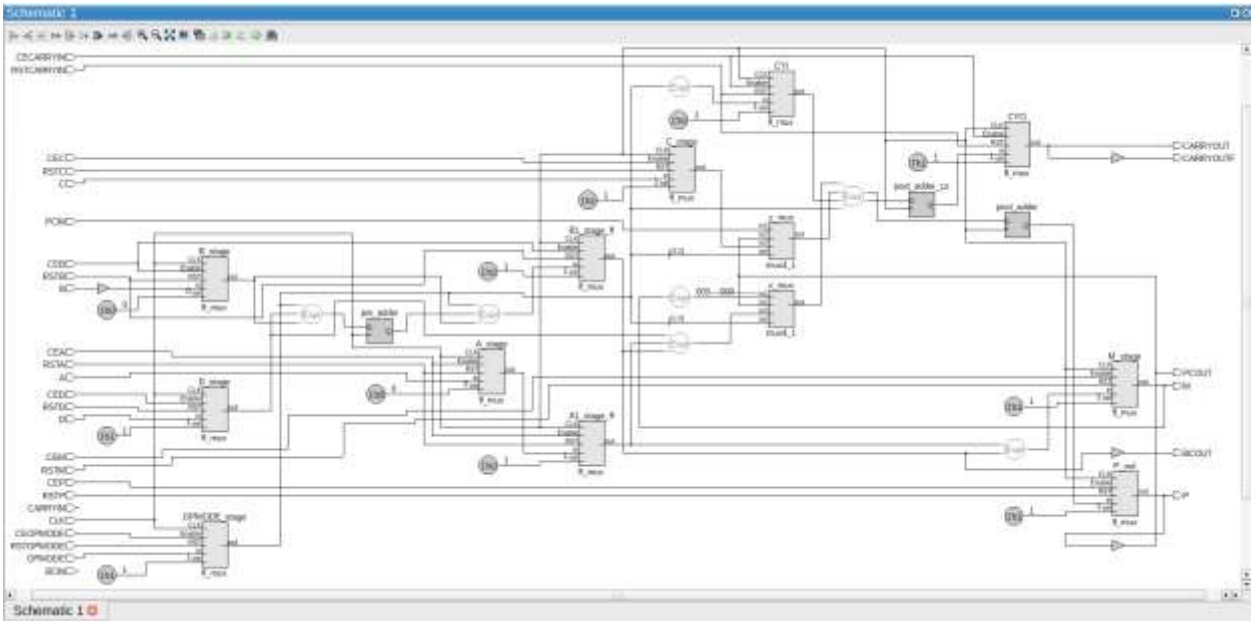

Figure 25 shows the generated Schematics in QuestaLint

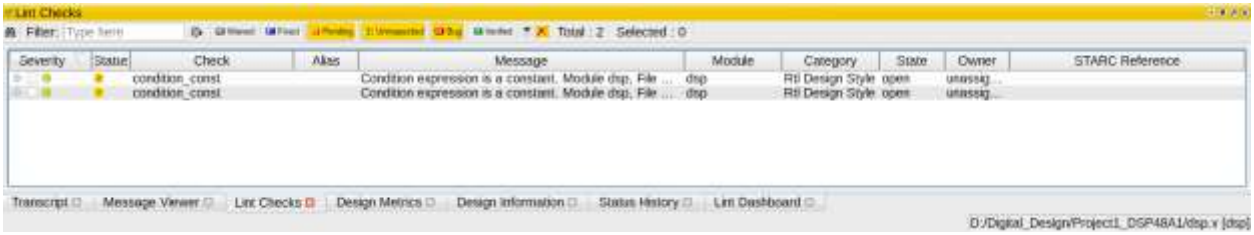

Figure 26 shows No Errors or Critical Warnings after Linting

FINAL OUTPUT: DSP48A1 MODULE AS SHOWN IN FIGURE 27

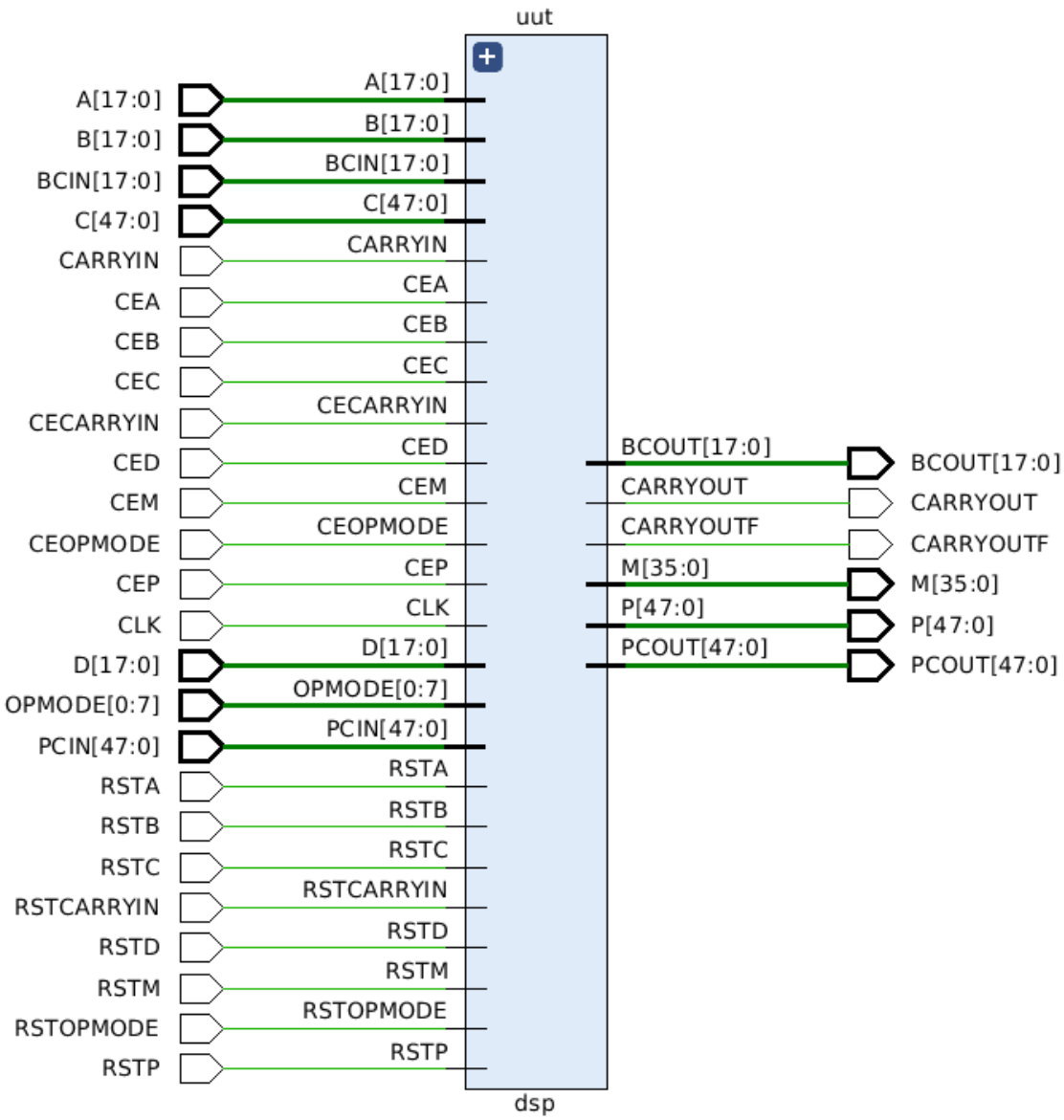

Figure 27 shows the DSP48A1 Module